

Game 01 — Tap or Don't Tap (Complete Plan)

Game 1 of 7 Complexity: ★ Low Estimate: 1–2 days Status: Mostly built

Complete plan (supersedes the 2026-09-09 sample). Status: **built & browser-verified** (2026-09-09 build pass) — `apps/frontend/public/games/tap-or-dont-tap/` with decoys, Stroop traps, rule swap, audio, pause and a baked percentile table. Logic tests: `apps/frontend/src/__tests__/games-tap-or-dont-tap.test.ts`.
Rev 2 (2026-09-11): architecture upgraded — pure `core.js`, `config.js`, `storage.js` (versioned + migrated), `audio.js`; analytics removed; a11y pass. 30/30 tests green. This doc is the spec of record: the constants below are authoritative when tuning, and §13 tracks the Rev 2 state.

1. Overview

A Go/No-Go reaction game (the real psychology self-control test). The shareable number is the **best reaction time in ms**. Success metric: median first session ≥ 15 rounds; players can state their ms score from memory after one session.

2. Complete game spec

Round loop (state machine: `MENU → WAITING → SIGNAL → FEEDBACK → (WAITING | GAMEOVER)`), plus `PAUSED`):

- WAITING** — dark screen, random delay **uniform 300–3000 ms**. A tap here is a **false start**: lose a heart, skip the round (no signal shown).
- SIGNAL** — one of:
 - GREEN → tap within the window.
 - RED → do not tap; survives after the window expires.
 - / ● DECOY (round ≥ 10) → never tap; ignoring it for the full window = +50.
 - STROOP TRAP (round ≥ 15) — word "TAP" in red or "WAIT" in green; **color wins, word ignored**.
- FEEDBACK** — 400 ms overlay ("218ms! 🔥" / "❌ Too slow" / "✅ Resisted +25"), then the next round begins. (Amended 2026-09-12 to match shipped tuning — no blank gap.)

Constants (authoritative — implemented in `game.js`)

Constant	Value	Notes
<code>WAIT_MIN/MAX_MS</code>	300 / 3000	uniform random
<code>WINDOW_START_MS</code>	1200	signal visible duration, round 1
<code>WINDOW_SHRINK_MS</code>	25	per round
<code>WINDOW_FLOOR_MS</code>	450	never below

Constant	Value	Notes
HEARTS	3	0 → game over
GREEN_BASE_POINTS	100	<code>points = min(100 + streak×10, 500)</code> (×5 cap)
RESIST_BONUS	25	red expired untouched
DECOY_BONUS	50	decoy expired untouched (round ≥ 10)
FEEDBACK_MS	400	overlay dwell, then next round (amended 2026-09-12)
<code>SWAP_FROM_ROUND</code> / <code>SWAP_CHANCE</code>	12 / 0.15	from round 12, 15 %/round; swap lasts <code>SWAP_ROUNDS</code> = 3 rounds — "📖 RULES SWAPPED!" (amended 2026-09-12 to match shipped tuning)

Signal mix per round band (amended 2026-09-12 to match shipped tuning): 1–9 → 55 % green / 45 % red; 10–14 → 45 green / 35 red / 20 decoy; 15+ → 40 green / 30 red / 15 decoy / 15 Stroop.

Losing: tap on red/decoy/Stroop-wrong, false start, or green missed → –1 heart each.
Reaction recorded only on green hits. Swap inverts green/red only; decoys always "never tap".

3. Progression & percentile model

- Difficulty = shrinking window + signal mix. No other knobs.
- **Best reaction** = min green-hit reaction of the run; **score** = sum of points.
- **Top-% badge**: `BAKED_TABLE` in `game.js` maps score → percentile (fixed simulated distribution). Personal history (last 20 runs `{score, bestMs}`) stored locally (amended 2026-09-12: cap 20, no per-entry date — matches shipped schema).

4. Screens & UI

State	Elements
menu	Title, one-line rule, best score + best ms, ▶ Start, mute toggle
waiting	Full-dark screen, "wait for it..." hint, HUD (hearts, score, round)
signal	Full-bleed color, optional overlay word (Stroop), HUD persists
feedback	Center overlay text + tone color, 400 ms
gameover	Card: score, best ms, rounds, top-% badge, 📦 Share, ↺ Retry, ☰ Menu
paused	Overlay "Paused — tap to resume" (auto on <code>visibilitychange</code>)

Whole screen is the tap target (`pointerdown`); `touch-action:none` on the play area;
Space/Enter equivalent on desktop; HUD height fixed across states (no relayout).

5. Data model (localStorage, guarded wrapper)

```
game:tap-or-dont-tap:best    → { score: 1240, bestMs: 187 }
game:tap-or-dont-tap:history → [{ score: 1240, bestMs: 187 }, ...max 20]
game:tap-or-dont-tap:muted   → true|false
```

6. File structure & function inventory

```
tap-or-dont-tap/
  index.html    # state sections + gameover card
  style.css     # flash colors, HUD, overlays
  game.js       # constants → pure core → state machine → DOM wiring
```

Pure core (no DOM — the test surface, currently in `game.js`, mirrored in the site test

file): `windowMs(round)`, `pointsForGreen(streak)`, `pickSignal(round, rng)`,
`resolveTap({signalKind, expectsTap, tapped, elapsedMs})`, `percentileFor(score)`.

DOM layer: `startRun`, `beginRound`, `showSignal`, `handleTap`, `expireSignal`,
`finishRound`, `loseHeart`, `gameOver`, `pauseRun`, `showFeedback`, audio helpers.

7. Edge cases (verify each once)

1. Hidden tab during WAITING → pause; on return restart the wait from 0. Hidden during SIGNAL → counts as a miss (anti-cheat).
2. Tap within 50 ms of a state change → single tap only (pointerdown debounce).
3. Private mode / storage disabled → game runs, persistence silently skipped.
4. Phone sleep mid-wait → `visibilitychange` → pause path.
5. Stroop × swap overlap (rounds 21–23) → swap inverts green/red; decoys never tappable.
6. Rapid restart from gameover → all pending timers cleared.
7. Reaction ≤ 0 ms or $>$ window + 50 ms → discarded as anomaly (clock sanity).

8. Testing plan

Existing: `apps/frontend/src/__tests__/games-tap-or-dont-tap.test.ts` — keep green.

Required cases (verify present; add missing): `windowMs` ramp incl. floor;

`pointsForGreen` cap; `resolveTap` truth table (5 signal kinds × tap/late/no-tap);

seeded `pickSignal` sequences per band; `percentileFor` monotonic + bounds.

9. Task breakdown

P0 — playable core — DONE (2026-09-09 build pass)

- ☒ State machine + round loop; signal kinds; hearts; reaction capture; ramp; scoring
- ☒ HUD + gameover card; pause/resume (`visibilitychange` incl. wait-restart)

P1 — full rules & feel — DONE

- ✓ Streak multiplier; decoys; Stroop traps; rule swap; false starts
- ✓ Feedback overlays; WebAudio + mute; baked percentile badge

P2 — persistence/share/QA

- ✓ Best score + history persistence; share chain
- ✓ **Delete** `analyticsCandidates` / `trackGamePlayed` **stubs** — done 2026-09-11 (Rev 2); the game makes zero network calls (isolation greps clean)
- ✓ Storage/audio/config helpers moved to per-game modules (`storage.js` , `audio.js` , `config.js`) per the Rev 2 architecture — supersedes the `../shared/` idea
- ✓ Pure core extracted to `core.js` ; site test file re-pointed and green (30 tests)
- ✓ QA gate (master README §5): offline play + isolation greps clean (2026-09-11)

P3 — polish

- ☐ Reduced-motion variant of flash; tuning pass after 5 test sessions
- ☐ History sparkline on menu; haptics on heart loss; sound design pass

10. Acceptance criteria

- §7 edge cases each verified once (note date when done); §8 tests green.
- 20 consecutive rounds, no stuck state; restart from gameover < 300 ms to first signal.
- Share works through all three fallback paths (share sheet / clipboard / prompt).

11. Deferred coupling (intentionally NOT built)

Footer/nav links, `game_played` analytics posting, achievements, leaderboard backend — per master README §7. The analytics stubs in `game.js` are to be **deleted**, not wired.

12. Remaining work (from code review 2026-09-10)

1. ~~Remove analytics stubs~~ — done 2026-09-11 (Rev 2 R2-1; isolation greps clean).
2. ~~Shared-utils migration + `core.js` extraction~~ — done 2026-09-11 as per-game `config.js` / `storage.js` / `audio.js` + `core.js` (Rev 2 R2-1; supersedes `../shared/`).
3. ~~Reduced-motion + tuning~~ — reduced-motion covered (flash is a solid color; pop + button transitions disabled); tuning pass still owes human sessions (P3).

13. Rev 2 architecture upgrade (2026-09-11) — reference: `03-sliding-puzzle.md`

Owner-approved architecture reference for all games (Sliding Puzzle Rev 2), applied per this game's nature: a reflex game has no levels and no meaningful multiplayer, so the upgrade here is **hygiene + structure**, not features. Isolation contract unchanged: no analytics, no site coupling, offline-first.

Target file layout

```
tap-or-dont-tap/
  index.html
  style.css
  config.js      # flags + per-locale strings (menu rule line, feedback/share/gameover
                 # copy), host-overridable: window.__TAP_OR_DONT_TAP_CONFIG__ → ?locale= → defaults
  storage.js     # guarded facade: versioned save + migrations, remote adapter slot
  audio.js       # WebAudio + mute
  core.js        # extracted pure core (windowMs, pointsForGreen, pickSignal, resolveTap,
                 # percentileFor) – the test surface; jest suite re-pointed here
  game.js        # UI shell: state machine + DOM wiring
```

config.js

Flags and strings only — no multiplayer, no levels in this game. Per-locale strings for the rule line, feedback overlays, share template and gameover card (shipped 2026-09-12 — 18 keys, all consumed); `prefs` may override the copy but is documented as a cache, not the source of truth.

storage.js facade

One versioned save record replaces the three loose keys:

```
game:tap-or-dont-tap:save → { version:1, best:{score,bestMs}, history:[...≤50], prefs:{muted} }
```

with a one-time migration from the legacy keys. Remote adapter slot (host-injected only) reserved for future account sync of best/history — the game never checks auth; ****guests keep full local persistence****. Mid-round resume: N/A (rounds are seconds).

Multiplayer (per this game's nature)

- Live/real-time duels: **gated** — device input/display latency makes reaction times unfairly comparable; needs a backend plus server-side timing and a \$7 reversal. Not planned.
- Optional future: **async seeded duel** — `pickSignal(round, rng)` already accepts an rng, so a shared seed fixes the identical signal sequence for both players; compare score. Deferred; not scheduled.

Phases

-  R2-1 Hygiene (2026-09-11): analytics stubs deleted; pure core extracted to `core.js` (jest suite re-pointed); `config.js` (locale + share strings, host-overridable) and `storage.js` (versioned `save` document, legacy-key migration, remote adapter slot) introduced; `audio.js` split out. 30/30 tests green; browser-verified end-to-end.
-  R2-2 A11y/polish (2026-09-11): signal `aria-label` describing each flash (incl. the Stroop lie) via `role="img"`; focus lands on Retry at gameover; `data-state` now maintained on the game root; Stroop word contrast fixed on the green flash; go-badge gradient darkened to AA; visible `:focus-visible` rings. (Reduced-motion: the flash is a solid color — nothing moves; the existing block covers the feedback pop + buttons. Sparkline + haptics were already shipped.)

- ☒ R2-1b Cleanup (2026-09-12, stale-code scan): unused `isMuted` / `whenSec` removed; history writes trimmed to the plan schema (`{score, bestMs}` — `rounds` / `ts` dropped); dead flash-yellow word rule removed; `defaultSave()` single-sourced; config strings completed (menu rule, swap banner, feedback overlays, gameover card); `FEEDBACK_MS` single-sourced. 31/31 tests green.
- ☐ R2-3 (deferred, optional) async seeded duel — only if there's demand.

Source: plan/games markdown · generated 2026-09-09 · part of the 2D Games sample plans (plan/games/pdf).

Game 02 — Tic Tac Toe (Complete Plan)

Game 2 of 7 Complexity: ★ Low Estimate: 1 day Status: Half built

Complete plan (supersedes the 2026-09-09 sample). Status: **built** — P0–P2 complete plus misère from P3; X/O theme picker skipped. `apps/frontend/public/games/tic-tac-toe/` (`index.html`, `style.css`, ESM `game.js`, `game.test.html`); jest twin at `src/__tests__/games-tic-tac-toe.test.ts` incl. the exhaustive minimax sweep. Constants below are the spec of record; §9–§11 list the only remaining work.

1. Overview

Classic tic tac toe: two players on one device, or single player vs. the computer at three AI levels. Hook is the running series score ("first to 5?") and a provably unbeatable Hard AI.

2. Complete game spec

- 3×3 board; **X moves first in round 1**; alternate marks each round; the **loser of a round starts the next**; after a draw, the alternating starter flips.
- Win = 3 in a row (8 lines: 3 rows, 3 cols, 2 diagonals). Draw = full board, no win.
- **Modes:** `2P local` (pass-and-play) · `1P` vs computer with levels:
 - **Easy** — uniformly random legal move.
 - **Medium** — win/block heuristic:
 1. take a winning move if one exists
 2. block an opponent win
 3. otherwise random legal move
 - **Hard** — full minimax (unbeatable); ties broken by first-found move in scan order (deterministic for tests).
- **Misère variant** (toggle): completing 3-in-a-row **loses**. Win detection is the same `checkWinner`; the `_interpretation_` flips (winner of a line loses the round). Draw rules unchanged.
- Illegal input impossible: occupied cell, finished round, or out-of-turn tap are no-ops; input locked during AI "thinking" tick (~300 ms artificial delay for feel).

3. Series model

Series = per-mode running tally. Persisted per `(mode, level)` key; Reset zeroes the current series only.

4. Screens & UI

State	Elements
menu	Mode pick (1P/2P), level pick (1P), misère toggle, series scoreboard, ▶ Play
playing	Board grid, turn indicator (mark + color), series mini-score, △ Menu
roundEnd	Overlay: "X wins! 🏆" / "O wins!" / "Draw 🟡" (misère wording flips), winning line
	highlighted through the 3 marks, [Next round] [Menu]

Board = CSS grid; marks render as inline SVG strokes with draw-in animation. Winning line gets a distinct highlight class. Fits 360 px viewport with no scroll.

5. Data model (localStorage)

```
game:tic-tac-toe:series:<modeKey> → { x: 3, o: 2, d: 1 } # modeKey = '2p' | '1p-easy' | '1p-medium' | '1p-hard'
game:tic-tac-toe:prefs              → { mode, level, misere, muted }
game:tic-tac-toe:best               → n/a (no score; share uses the series line)
```

(2026-09-12: the Rev 2 `storage.js` migration accepts **both** this per-modeKey layout and a single `game:tic-tac-toe:series` key with colon setupKeys — tallies are remapped (`d` → `draw`) and merged field-wise, so a pre-Rev-2 save in either layout survives.)

6. File structure & function inventory

```
tic-tac-toe/
  index.html      # menu / playing / roundEnd sections
  style.css       # grid, marks, win highlight, overlays
  game.js         # ESM: pure logic exported + DOM wiring
  game.test.html  # browser twin of the jest suite
```

Pure exported logic: `newBoard()`, `applyMove(board, i, mark)`, `checkWinner(board) → { winner, line } | 'draw' | null`, `legalMoves(board)`, `minimaxScore(board, mark)`, `bestMove(board, mark)` (hard), `mediumMove(board, mark)`, `easyMove(board, mark, rng)`, `resolveMisere(result)`. DOM layer: screens, `renderBoard/renderCell`, `highlightWin(line)`, `isAiTurnNow()`, series storage, share.

7. Edge cases

1. occupied / finished / out-of-turn taps = no-ops; double-tap debounce 50 ms.
2. AI never acts during `roundEnd`; Next-round resets board but not series.
3. Misère + AI: Hard minimax optimizes the `_misère_` objective (inverted leaf scores) — covered by the exhaustive test at this variant too.

- 4. Storage disabled → defaults, no persistence, no crash.
- 5. Series key changes when mode/level changes (no cross-mode pollution).
- 6. Draw starter alternation persists only within a session (not in storage) — stated.

8. Testing plan (jest twin + game.test.html)

- `checkWinner`: all 8 lines, draw, empty, mid-game null.
- Hard AI: exhaustive sweep — never loses from any reachable position (both normal and misère objectives).
- Medium: always takes an available win; always blocks an immediate threat.
- Series persistence round-trip; mode-key isolation.
- Misère interpretation flips round outcome without changing `checkWinner`.

9. Task breakdown

P0/P1/P2 — DONE (2026-09-09/10 build pass)

- ☒ 2P local, win/draw detection, win-line highlight, round flow, series scoreboard
- ☒ 1P Easy/Medium/Hard (minimax), misère variant, SVG marks + animations
- ☒ Persistence, tests (incl. exhaustive unbeatable sweep), game.test.html

P2 — remaining

- ☒ **Resolve analytics coupling** — done 2026-09-11 (Rev 2 R2-1): the POST block was deleted; the game makes zero network calls (isolation greps clean).
- ☒ QA gate (master README §5): offline play + isolation greps clean (2026-09-11).

P3 — remaining polish

- ☒ Misère variant — done
- ☒ Board flip animation — done
- ☐ X/O theme picker (skipped by owner decision — keep skipped unless requested)

10. Acceptance criteria

- §7 cases verified; §8 tests green in jest and browser twin.
- Hard AI unbeatable in exhaustive sweep (both objectives).
- Series survives reload; Reset works; 360 px fits with no scroll.

11. Deferred coupling (intentionally NOT built)

Online multiplayer (needs backend rooms), footer/nav links, Play Hub card, achievements — per master README §7. Existing analytics POST block is the one live violation; flagged above, not silently removed.

12. Rev 2 architecture upgrade (2026-09-11) — reference: 03-sliding-puzzle.md

Owner-approved architecture reference for all games (Sliding Puzzle Rev 2), applied per this game's nature: tic tac toe is turn-based, so its multiplayer story is "hot-seat is already shipped; async position challenges are a cheap future add; live online stays gated." The upgrade is otherwise persistence hygiene + structure.

Target file layout

```
tic-tac-toe/
  index.html
  style.css
  config.js      # flags + per-locale strings (round-end/series copy, AI think-delay),
                 # host-overridable: window.__TIC_TAC_TOE_CONFIG__ → ?locale= → defaults
  storage.js     # guarded facade: versioned save (series tallies + prefs) + migrations,
                 # remote adapter slot (host-injected only)
  core.js        # extracted pure logic (board, checkWinner, minimax/medium/easy, misère)
                 # – the test surface; jest suite + game.test.html re-pointed here
  game.js        # UI shell: screens, rendering, series wiring
  game.test.html
```

config.js

Constants currently living in `game.js` (AI think-delay ms, mark order) plus per-locale strings for round-end and series copy. `prefs` may override copy but is a cache, not the source of truth.

storage.js facade

One versioned save document (series tallies + prefs together, `version` field) migrated from the loose series keys — both the per-modeKey layout of §5 and the single-key layout are accepted and merged (2026-09-12). Remote adapter slot reserved for future account sync of series — the game never checks auth; **guests keep full local persistence**.

Mid-round resume: N/A (rounds are fast by design).

Multiplayer (per this game's nature)

- Hot-seat 2P: **already shipped** (pass-and-play + series scoreboard).
- Future/optional: **async position challenge** — share a mid-game position + turn as a link ("can you win from here?"); a pure function of the board, no backend.
- Live online rooms: **gated** — backend + §7 reversal (§11). Not planned.

Phases

- ☒ R2-1 Hygiene (2026-09-11): analytics POST block deleted (greps clean); pure logic extracted to `core.js`; `config.js` (locale, AI think-delay, share templates, host-overridable) and `storage.js` (versioned `save` document migrating the loose series/prefs keys, remote adapter slot) introduced; jest suite + `game.test.html` re-pointed (23 tests green incl. the exhaustive minimax sweep).
- ☒ R2-2 A11y (2026-09-11): already largely present (roving board focus, per-cell labels, roundEnd focus on Next); verified — no further changes needed.
- ☐ R2-3 (deferred, optional) async position challenge — only if there's demand.

Source: plan/games markdown · generated 2026-09-09 · part of the 2D Games sample plans (plan/games/pdf).

Game 03 — Sliding Puzzle (Complete Plan)

Game 3 of 7 Complexity: ★★ Medium Estimate: 1–2 days Status: Built

Revision 2 (2026-09-11, owner-approved). The game is built and live (P0–P2 plus 5×5, picture modes, 8 procedural scenes, corner target preview, daily challenge, hard mode). This revision adds the approved **upgrade roadmap (Phases 0–5)**: a modular file structure, campaign levels + free play, a curated picture library with a builder tool, same-device + own-device multiplayer behind a config flag, and local-first persistence with mid-round resume. Supersedes the 2026-09-09 sample and revision 1.

1. Overview

Classic 15-puzzle: slide tiles into order. Hook is the personal-record pair ("4×4 in 2:41 · 213 moves"), guaranteed-solvable shuffles, and picture rounds assembled from a target image shown in the board's corner. Goal of this revision: make the game a **flagship, future-proof title** — more content (pictures, levels), more modes (daily, hard, multiplayer), and an architecture that grows without rework, while keeping the repo's constraints: vanilla ESM, no build step, no dependencies, offline-capable, games isolated from the website product.

2. Game spec (live today)

- **Sizes:** **3×3** (default), **4×4** (classic), **5×5**. Tiles $1..N^2-1$, blank last.
- **Shuffle:** from solved, `movesCount` random legal single-tile slides — 120 (3×3), 250 (4×4), 400 (5×5) — never undoing the previous move. Solvable by construction; inversion-parity (`isSolvable`) is verified in tests anyway.
- **Move counting:** single-tile slide = 1 move; row/column segment push = 1 move per displaced tile.
- **Timer:** starts on first move, stops on the winning move, accumulates across pauses (`performance.now()` only), never runs in `menu` / `won`.
- **Win:** tiles in order, blank last → overlay. Input locked after the winning move.
- **Score:** `max(0, sizeBonus - ⌊moves/2⌋ - ⌊seconds/5⌋)`, sizeBonus 300/500/800.
- **Wrong-tile feedback:** shake animation, no move counted.
- **Modes (live):** Numbers or Picture tiles; **Daily challenge** (seeded 4×4 — same board for everyone that day, per-day record); **Hard** (picture-only, no preview/peek, separate records).

3. Progression model (this revision)

- **Campaign:** sequential stages with difficulty ramping (grid size, shuffle depth, fading assists, par moves/time). A stage unlocks when the previous is solved. Stars (1–3) from par thresholds; replays improve stars. Per-stage best time/moves.

- **Free Play:** today's menu verbatim (any size, numbers/picture, daily, hard) — always available, **no unlocks, no stars, no campaign progress** (its limitation). Per-size/mode records still apply.
- **Daily:** unchanged — seeded 4×4, per-day record; picture becomes deterministic from the same seed (planned, Phase 1) so every player sees the identical board _and_ image.
- **Records:** per size/mode/variant (`best:<size>[:hard]`), per day (`daily:<yyyymmdd>`), and per campaign stage (`progress`), all localStorage, schema-versioned.

4. Screens & UI (this revision)

Screen	Elements
Menu (hub)	Section entry points: Campaign, Free Play, Multiplayer ; mute; theme follows site
Campaign	Stage grid: lock state, stars, best time/moves per stage; continue point
Free Play	Today's menu: size picker (3 cards with bests), numbers/picture, hard, daily (with date + done state)
Playing	Compact centered cluster: HUD (menu · timer · moves · pause), board (<code>--n</code> -driven), corner target preview (picture rounds), actions (peek · restart · shuffle)
Paused	Opaque overlay — board hidden, clock held; resume/restart/menu
Won	Time, moves, score; NEW BEST badges; campaign variant adds stars + [Retry] [Next stage]
Multiplayer	Series (same device) and challenge link (own device); entry gated by flag (§7)

Tiles are absolutely-positioned buttons moved via `transform: translate` transitions (180 ms ease); the blank is a missing tile. Board sizing `min(92vw, 60dvh)`, capped for desktop; HUD + board + actions cluster together at every viewport.

5. Data model (localStorage, schema-versioned, guarded)

```
game:sliding-puzzle:prefs      { version, size, muted, mode, hard, locale? }
game:sliding-puzzle:best:<size>[:hard]  { timeMs, moves }
game:sliding-puzzle:daily:<yyyymmdd>    { timeMs, moves }
game:sliding-puzzle:progress      { version, unlockedUpTo, stages:{ id:{timeMs,moves,stars} } }
game:sliding-puzzle:resume       { version, size, seed?, board, moves, playedMs, mode, assists, stageId? }
```

All reads validate + migrate by `version`; all writes best-effort (storage disabled → game fully playable). A pluggable **remote adapter slot** exists in `storage.js` for a future host-injected account sync — the game never checks auth and makes no network calls (§7).

6. File structure (modular — replaces the original 4-file layout)

```
sliding-puzzle/
  index.html      screens: menu hub | campaign | free play | playing | multiplayer
  style.css       tokens (light/dark), screens, board, overlays, reduced-motion
  core.js         pure model: board, shuffle (+guards), slideTile (+displaced tiles),
                  solvability, scoring, clock format, per-size config, key builders,
                  mulberry32, mergeRecord — no DOM (the test surface)
  levels.js       [P1] stage definitions + unlock/star rules — pure
  scenes.js       [P0] procedural painters + makePicture + sliceBackground
  library.js      [P2] picture-library manifest access — pure
  config.js       [P0] flags + strings, host-overridable (§7)
  storage.js      [P0] guarded storage facade: version + migrations, prefs, records,
                  progress, resume, remote adapter slot
  audio.js        [P0] WebAudio blips + mute (context on first gesture)
  game.js         UI shell: screens, input, timer, share (imports the modules above)
  data/pictures.json [P2] library manifest (empty until packs are added)
  pics/           [P2] bundled square images (only when the owner adds them)
  builder.html    [P2] Library Builder tool (local, not linked from the game)
  core.test.html  in-browser harness for the pure modules (auto-runs)
```

7. Multiplayer & the website/app flag

Modes (both backend-free):

- **Same-device series:** two players alternate on identical seeded boards; per-round result + series scoreboard (Tic Tac Toe's pass-and-play pattern). Separate records.
- **Own-device challenge:** a challenge encodes `{size, seed, assists, pictureId}` in a URL/code; the friend plays the `_identical_` board on their device; results compared via share text + a manual "opponent result" entry. Nothing at stake → no trust problem.

Website/app flag (config.js):

```
multiplayerEnabled: false // website default; the app wrapper injects true
```

Resolution order: host-injected `window.__SLIDING_PUZZLE_CONFIG__` → `?mp=0|1` URL param

→ default. One file, no refactor to switch environments (the flag arrives when that static file is re-served/injected — cache-busting in §12).

- `true` → multiplayer behaves normally.
- `false` → the Multiplayer entry stays **visible but disabled**: click opens a small dialog with the configurable notice ("Multiplayer available in the app only."), rendered `aria-disabled`; **no multiplayer logic executes, no network calls**. Modules stay loaded but inert. Notice text lives in `config.strings` (per-locale); `prefs` may override it but is a cache, not the source of truth.

Gated — explicitly NOT built (needs backend + \$7 reversal): live rooms, real-time races, in-app leaderboards, account-level sync. Client `performance.now()` cannot be trusted competitively; any of these requires server-side timing and an explicit, separately-scoped owner change reversing the isolation decision.

8. Persistence (local-first)

- **Guests and the website build: full local persistence** — records, daily, campaign progress, resume. Dropping guest saving would erase most players' progress and break the offline contract; not done.
- **Resume:** an unfinished round snapshots `{size, seed?, board, moves, playedMs, mode, assists, stageId?}` (~200 B) and offers "Continue" on return — for everyone, guests included.
- **Logged-in continuity is a host concern:** `storage.js` exposes a remote adapter slot; the website/app injects an adapter for authenticated users (local-first, best-effort mirroring). The game never checks auth → no coupling, no network in the website build.
- **Account-level/cross-device sync — gated future phase (\$7):** needs the backend and an explicit isolation reversal. Documented, not built.

9. Testing

- **core jest suite** (`src/__tests__/games-sliding-puzzle.test.ts`): model proofs — 100 shuffles/size solvable + never solved + no-undo replay through `slideTile`; segment slides; solvability formulas (odd/even width); scoring; seeded daily determinism; record merging; plus suites for the new pure modules: per-size config resolution, shuffle guards, levels (unlock/star boundaries), storage migrations, library manifest parsing/fallback, config flag resolution, `sliceBackground`.
- **core.test.html** harness mirrors the pure assertions in-browser (auto-runs).
- **UI layer** (`game.js`) is verified by browser passes (flag states, layouts at 360 px and desktop, resume accuracy ± 50 ms) — UI logic stays thin by pushing rules into the pure modules above.
- CI safety: every committed test imports only tracked files.

10. Build phases (approved roadmap)

Phase 0 — Foundation: correctness fixes, config/flags, storage facade

- ☐ `shuffle()` guards: validate `movesCount` (unknown size → clear error), bound the solved-retry loop (no infinite loop from a degenerate rng); pathological case falls back to a forced nudge.
- ☐ One per-size config object (`shuffleMoves`, `sizeBonus`) as the single source; `SIZES / SHUFFLE_MOVES / SIZE_BONUS` derived from it; unknown size fails fast.
- ☒ `slideTile` returns displaced tile values; UI stops re-implementing the geometry. (2026-09-12: returns `{ board, moved, pushed }`; `pushedTileValues` deleted.)
- ☒ `config.js` + `storage.js` + `scenes.js` + `audio.js` modules created; `game.js` becomes the UI shell importing them (behavior unchanged). (2026-09-12)
- ☒ Dead CSS removed (`board--deal`, `overlay--in`, `@keyframes deal/pop`); `.daily` added to reduced-motion. (2026-09-12)
- ☐ Score/clock rounding unified.

- ☒ Jest + harness updated for the new modules; all green; game behavior unchanged. (2026-09-12: versioned `game:sliding-puzzle:save` v1 with legacy-key migration.)

Phase 1 — Campaign levels + Free Play

- ☐ `levels.js`: stage definitions (~24 stages, three tiers: 3×3 → 4×4 → 5×5, assists fading, fixed picture per stage, par + star thresholds) and pure rules (`stageById`, `isUnlocked`, `starsFor`, `nextStage`).
- ☐ `storage.js`: progress record (versioned, migrated) + free-play separation.
- ☐ UI: Campaign screen (grid, locks, stars, bests), Free Play screen (today's menu), campaign win overlay (stars, retry/next), hub navigation.
- ☐ Daily picture deterministic from the daily seed.
- ☐ Jest: unlock/star/scoring boundaries; browser: unlock flow + free-play parity.

Phase 2 — Picture library + Library Builder

- ☐ `data/pictures.json` manifest schema (packs, pictures, sections) + `library.js` (resolve/fallback) + `scenes.js` picture resolution for levels/daily/free.
- ☐ `builder.html`: upload → square-crop/resize 720×720 → JPEG q0.85 → assign packs + sections → export `pictures.json` + named `pics/<pack>/<id>.jpg` files.
- ☐ Rules: ≤ ~150 KB per image, square, license-clear (owner's responsibility); zero entries → procedural-only behavior unchanged.
- ☐ Jest: manifest validation/resolution/fallback; browser: assigned image slices correctly + appears in the corner preview; builder round-trip.

Phase 3 — Multiplayer + website/app gating

- ☐ Same-device series: alternating seeded boards, per-round results, series scoreboard.
- ☐ Own-device challenge: URL/code encoding `{size, seed, assists, pictureId}`; identical board + picture on the friend's device; result comparison via share + manual opponent entry.
- ☐ Flag gating per §7: disabled entry + dialog + zero logic/network; `?mp=1` /host override enables full behavior.
- ☐ Jest: challenge encode/decode round-trip; browser: both flag states, network-request log empty when disabled; isolation grep clean.

Phase 4 — Persistence & resume

- ☐ `storage.js` migrations + remote adapter slot (host-injected only).
- ☐ Mid-round resume snapshot (save on state change, offer Continue on return); works for guests; campaign rounds resume into their stage.
- ☐ Docs: host-injection contract for the adapter (the game never checks auth).
- ☐ Jest: snapshot save/restore round-trip, migration path; browser: resume accuracy ±50 ms; zero network calls with no adapter injected.

Phase 5 — Polish, accessibility, QA, docs

- ☐ A11y: tile position in `aria-label` + blank announcement (polite live region), focus trap/inert for `aria-modal` dialogs, roving focus + arrow keys for radio groups, focus target when a round starts.

- ☐ Contrast to AA (primary gradient light stop, muted text); `color-mix` fallback + `-webkit-backdrop-filter`.
- ☐ Cache-busting for `style.css` / `game.js` / `data/pictures.json`.
- ☐ Timer/share helper extraction + tests; determinism test fixed (two fresh streams); mid-board segment + non-integer index cases.
- ☐ Docs: this file's status + `plan/games/README.md` row per phase; gated items (§7) recorded.
- ☐ QA gate (master README §5): offline check, isolation greps, 10-minute phone session (incl. iOS Safari fallbacks).

11. Acceptance criteria

- All Phase checklists above checked; jest + `core.test.html` green; CI safe (tests import only tracked files).
- Shuffle can never hang regardless of config or rng; unknown sizes fail fast with a clear message.
- `multiplayerEnabled:false` (website default) = single-player only, zero network calls, verified in the browser's request log.
- Timer exact across pause/resume (± 50 ms); resume restores board/moves/clock exactly.
- Campaign unlock/star rules correct at boundaries; Free Play never awards progression.
- Library: zero entries → today's behavior; assigned images slice correctly and appear in the corner preview; builder round-trips assignments.
- 3×3 fits 360 px with HUD visible, no scroll; layouts compact at phone and desktop.
- Restart ≤ 1 tap; shuffle visibly mixes the board every time.

12. Deferred coupling & gated items (intentionally NOT built)

- Footer/nav links, analytics events, achievements integration, backend leaderboards — per master README §7. Sliding Puzzle adds **no** analytics and **no** site coupling.
- **Gated future phases (require backend + explicit §7 reversal):** live rooms/real-time races, in-app leaderboards, account-level cross-device sync. Server-side timing validation is mandatory for anything competitive.
- Shipping the games publicly (removing `.gitignore` isolation lines + footer/sitemap/ Play Hub references) remains a separate, explicitly-scoped owner action. Sliding Puzzle is the only committed game (owner decision 2026-09-11).

Game 04 — Word Puzzle (Complete Plan)

Game 4 of 7 Complexity: ★★ Medium Estimate: 2–3 days Status: Built

Complete plan (supersedes the 2026-09-09 sample). Status: **Built** (P0–P2 complete, 2026-09-11) — full game in `apps/frontend/public/games/word-puzzle/` (`core.js/game.js/style.css` + `data/themes.json`), jest suite + `core.test.html` green (incl. the 100-seeds-per-level generator proof and data lint), browser smoke test covered drag / tap-tap / keyboard paths, hints, stars, progress and the theme-complete flow; P3 extras (daily puzzle, confetti, more themes) deferred. Interpretation decided: **word search** (master README §8 #1–2). Slug `word-puzzle`; folder `apps/frontend/public/games/word-puzzle/`.

1. Overview

Themed word search: drag a line through letters to find hidden words; correct finds lock in with colored highlights, wrong picks flash red. Hook: 3-star levels and the daily seed (P3) that gives everyone the same puzzle.

2. Complete game spec

- **Content:** 4 themes (Animals 🐾, Food 🍎, Tech 💻, Space 🚀) × 3 levels, shipped as static `data/themes.json` (schema §5). English, lowercase, 4–9 letters, no spaces, no duplicate words within a level.
- **Grid tiers:** L1 = 8×8, 5 words, directions {→, ↓}; L2 = 10×10, 7 words,
 - {↖, ↗}; L3 = 12×12, 10 words, all 8 directions incl. reversed.
- **Selection:** pointerdown on a cell anchors; pointermove samples cells via `document.elementFromPoint`, snapped to the grid; a valid drag follows one of the 8 direction vectors in a straight line. Live highlight while dragging; on release:
 - Letters match an unfound word (or its reverse where reversed dirs are on) → **locked** in the word's color, struck off the list, +pop animation. ✅
 - Otherwise → red flash 300 ms, **wrong picks +1**, selection clears. ❌
- **Hints:** 3 per level; a hint rings the first letter of a random unfound word.
- **Stars:** ★ finish the level · ★ time ≤ par · ★ `hintsUsed === 0 && wrongPicks ≤ 2`.
- **Par times:** L1 90 s · L2 150 s · L3 240 s (per level data, tunable).
- **Level flow:** complete → result overlay → next level → theme complete → badge. Progress persisted per level (§5).

3. Generator (pure, deterministic)

`generateLevel(levelData, seed)` using `mulberry32(seed)` :

1. Sort words longest-first. For each word: try up to 200 random placements (random direction from the tier's set, random start cell) — legal if in bounds and every cell is empty or holds the same letter (shared-letter crossings allowed).
2. If a word fails 200 attempts → restart the whole level (max 20 restarts, then throw — data bug, must never ship).
3. Fill empty cells from a letter bag: 60 % weighted toward the level's own letters, 40 % uniform A–Z (keeps grids solvable-feeling and dense).
4. Return `{ grid, placements: [{word, row, col, dir}], seed }`.

4. Screens & UI

State	Elements
menu	Theme cards (emoji, name, stars x/9, badge when complete), Continue banner, mute toggle
level	Letter grid + word list (chips; found = struck, colored), HUD: timer, hint stars, [Hint] [△]
result	Overlay: time, stars, words found, [Next level] [Replay] [🔖 Share]; theme-complete variant

Word list sits below the grid at phone width (side-by-side ≥ 768 px). Grid cells sized `min(92vw, 100% / size)`; touch highlight uses per-word colors from a fixed 10-color accessible palette (4.5:1 on white).

5. Data model

```
data/themes.json:
{ "themes": [ { "id": "animals", "name": "Animals", "emoji": "🐾",
  "levels": [ { "size": 8, "parSec": 90, "words": ["LION","ZEBRA", ...] }, ... ] }, ... ] }

localStorage:
game:word-puzzle:progress → { "<themeId>:<lvl>": { stars: 0-3, bestTimeMs: 84200 } }
game:word-puzzle:prefs    → { muted: false }
```

6. File structure & function inventory

```
word-puzzle/
  index.html
  style.css      # grid, highlight pills, flash animations
  core.js        # generator + validation (pure, seeded) – test surface
  game.js        # input, rendering, timer, stars, persistence
  data/themes.json
```

`core.js` exports: `mulberry32(seed)`, `generateLevel(level, seed)`, `lineCells(grid, start, end) → cells | null` (8-direction check), `lettersAt(grid, cells) → string``,

`matchesWord(letters, word, allowReverse)`, ``starsFor(parSec, timeSec, hintsUsed, wrongPicks)` . `game.js`: pointer/tap-tap/keyboard input, highlight rendering, timer, progress, share.

7. Edge cases

1. Diagonal reverse drags (e.g. ✓ when reversed on) must validate as the word reversed.
2. Drag that leaves the grid → clamp to last valid cell; release outside → evaluate.
3. Re-finding a found word → counts as correct-but-noop (no star penalty, no duplicate).
4. Wrong selection on cells that partially spell a word (prefix) → still wrong (full word required) — documented in-game via the red flash.
5. Tap-tap mode: first tap anchors (cell pulses), second tap submits; tapping the anchor again cancels.
6. Keyboard: arrows move focus; Enter anchors/submits; grid is one tab stop (roving focus), `aria-label` per cell = "row r column c, letter X"; word chips labeled "word k, N letters" (never the word itself).
7. `visibilitychange` pauses the timer; level restarts only via user action.
8. Storage disabled → progress not saved, everything else works.

8. Testing plan (core.js)

- Seeded determinism: same seed → identical grid/placements (snapshot).
- Every word present in `placements`, within the tier's allowed directions.
- Generated grid: all placements verified by `lineCells` + `lettersAt` round-trip.
- Generator termination: 100 seeds per shipped level, zero throws.
- `lineCells`: 8 directions, out-of-bounds, non-collinear → null.
- `matchesWord`: exact, reverse (allowed vs not), case handling.
- `starsFor`: boundary table (par ±1 s, hints 0/1, wrong 2/3).

9. Task breakdown

P0 — playable core

- ✓ `core.js`: mulberry32, generateLevel, lineCells, lettersAt, matchesWord + tests
- ✓ Grid rendering + drag selection with live highlight and correct/wrong resolution
- ✓ Word list check-off, level timer, win detection, next-level flow

P1 — full rules & feel

- ✓ 4 themes × 3 levels data (`themes.json`) + data lint (lengths, dupes, charset)
- ✓ Reversed directions at L3; hints; star rating; color pills; red flash; pop anim
- ✓ Progress persistence + Continue; sound blips + mute

P2 — persistence/share/QA

- ✓ Tap-tap + full keyboard path; a11y labels (\$7.5–7.6)

- ☒ Share text (`{words} words in m:ss · ★★`); QA gate (master README §5) (browser smoke test at 390×844 covered seeded drags, tap-tap branches, keyboard, hints, stars, persistence, theme completion + lint harness; a human 10-minute session is still owed before calling it player-proof)

P3 — polish

- ☐ Daily puzzle (seed = `YYYYMMDD` hash → same grid for everyone, share-friendly); confetti on theme completion; more themes `_`(deferred — `?seed=` QA hook already ships the deterministic-generator half)

10. Acceptance criteria

- §8 tests green; every shipped level solvable and every word placeable (generator proof).
- Wrong picks never permanently mark letters; found letters stay locked.
- Drag usable at 360 px without zoom; grid + chips visible during a drag.
- Seeded daily mode reproduces identical grids across reloads.

11. Deferred coupling (intentionally NOT built)

Riddle/content-pipeline integration (backend word lists), footer/nav links, analytics events, achievements — per master README §7.

12. Rev 2 architecture upgrade (2026-09-11) — reference: `03-sliding-puzzle.md`

Owner-approved architecture reference for all games (Sliding Puzzle Rev 2), applied per this game's nature: word search is already data-driven (`themes.json`) and seeded (`mulberry32` + the `?seed=` hook), so its Rev 2 is persistence hygiene plus **promoting the daily/seed features that make async multiplayer natural.**

Target file layout (additions to the shipped trio)

```
word-puzzle/
  index.html
  style.css
  core.js      # generator + validation (unchanged)
  game.js     # UI shell (unchanged behavior)
  config.js   # flags (dailyEnabled) + per-locale strings (share template, hint copy,
              # theme-complete lines); host-overridable
  storage.js  # guarded facade: versioned progress + prefs + migrations,
              # remote adapter slot (host-injected only)
  data/themes.json
```

config.js

Per-locale strings (share template, hint wording, toasts, theme-complete lines).
(The placeholder `dailyEnabled` flag was removed 2026-09-12 — daily stays deferred with

P3; strings extraction completed 2026-09-12.) Host override

`window.__WORD_PUZZLE_CONFIG__` → `?locale=` →

defaults. `prefs` may override copy but is a cache, not the source of truth.

storage.js facade

Versioned progress record — `{ version, levels: { "<theme>:<lvl>": {stars, bestTimeMs} }, prefs }` — migrated from the loose per-level keys. Remote adapter slot reserved for future account sync of stars/progress — the game never checks auth; ****guests keep full local persistence****. Mid-round resume: N/A (levels are short; stars already persist).

Multiplayer (per this game's nature)

- **Async seeded race — the natural fit; promote from P3:** the generator is already deterministic (`generateLevel(level, seed)`, `?seed=` hook ships). Daily seed (`YYYYMMDD`) = everyone gets the same grid; a shared seed link = challenge a friend; compare time/stars. Hot-seat: pass the device per level.
- Live co-op/race: **gated** — backend + \$7 reversal. Not planned.

Phases

- ☒ R2-1 Hygiene (2026-09-11; strings completed 2026-09-12): `config.js` (locale, share template + hint/toast/theme-complete copy, host-overridable) + `storage.js` (versioned `save` document — levels + prefs — migrated from the loose progress/prefs keys, corrupt entries sanitized, remote adapter slot); strings moved out of `game.js`; 43 tests green.
- ☒ R2-2 (partial 2026-09-11): the deterministic generator + `?seed=` hook ship; the full daily UI (menu entry + per-day record, seed = `YYYYMMDD`) remains deferred with P3 (the placeholder `dailyEnabled` flag was removed 2026-09-12 until the mode ships).
- ☐ R2-3 Content: more themes via `themes.json` (data-only); confetti polish.

Game 05 — Continuous Runner / Hurdles (Complete Plan)

Game 5 of 7 Complexity: ★★ Medium Estimate: 2–3 days Status: Not started

Complete plan (supersedes the 2026-09-09 sample). Status: **not started** — build-ready.
Slug `hurdle-runner` ; folder `apps/frontend/public/games/hurdle-runner/` .
Its engine is the foundation for Spirit Runner (07) — build to the structure in §6.

1. Overview

Endless runner: the world scrolls, hurdles approach, you jump. One input, instant restart, meters as the bragging number. Success metric: median first session ≥ 3 runs; death always reads as the player's fault.

2. Complete game spec

World & camera

- Virtual viewport 900×500 px logical, scaled to fit (letterboxed, `devicePixelRatio` aware).
- Ground line at $y=420$. Parallax: far hills $\times 0.2$, near trees $\times 0.5$, ground $\times 1$ of scroll speed.
- Player fixed at $x=225$ (25 %).

Physics (fixed timestep 1/120 s, accumulator, 250 ms frame clamp)

Constant	Value	Notes
<code>SCROLL_START</code>	320 px/s	+6 px/s per second, cap 900
<code>JUMP_VY</code>	−820 px/s	on jump
<code>GRAVITY</code>	2200 px/s ²	
<code>JUMP_CUT</code>	0.55	releasing early caps upward velocity at 55 %
<code>COYOTE_MS</code>	80	grace after leaving ground
<code>BUFFER_MS</code>	100	early tap counts when landing
<code>HITBOX_INSET</code>	15 %	forgiveness on the player AABB

Obstacles (spawned ahead at $x=950$, culled behind $x=-100$)

Type	Size (w×h)	Introduced	Notes
Hurdle	24×40–60 px	0 m	low jump
Tall barrier	28×90 px	500 m	early, full-height jump

Type	Size (w×h)	Introduced	Notes
Double hurdle	two hurdles 140–180 px apart	1500 m	forces full-speed chained jumps

- **Spawner fairness:** gap between obstacles sampled 380–700 px, but never below `minGap(speed) = speed × 1.05 + 120` px (a full jump always fits). Doubles count the inner spacing toward the next gap too.
- **Collision:** AABB, player box ×0.85 inset; obstacle boxes full. Death = game over (one hit — decision master README §8 #9).
- **Score:** meters = px/10 (distance), +10 per obstacle cleared (its right edge passes player x).  pickup (P2): once per run, spawns 600–900 m; grants one extra life → on fatal hit, consume heart + 1 s invulnerability flash.

Tiers (HUD announces "Speed up!")

- Tier 1: 0–500 m (hurdles only) · Tier 2: 500–1500 m (+tall) · Tier 3: 1500 m+ (+doubles).

3. Screens & UI

State	Elements
<code>menu</code>	Title, best distance, control hint,  Run, mute toggle
<code>playing</code>	Canvas world; HUD: distance, next-tier progress bar, hearts (only if  held)
<code>paused</code>	Overlay (auto on <code>visibilitychange</code> / )
<code>gameover</code>	Distance, obstacles cleared, best badge ("NEW BEST!"), [ Retry] [ Share]

`?debug=1` query flag draws hitboxes and spawner-gap markers (dev aid, documented).

4. Controls

- **Jump:** tap anywhere / Space / ↑. `pointerdown`; hold = higher (jump cut on release).
- Coyote + buffering exactly as specced (feel-critical, in `Player.update`).
- Nothing else at MVP — one primary action. Down/fast-fall is P3.

5. Data model (localStorage)

```
game:hurdle-runner:best  → { distanceM: 1204 }
game:hurdle-runner:prefs → { muted: false }
```

6. File structure & function inventory (engine is 07's base)

```
hurdle-runner/  
  index.html  
  style.css  
  
  core.js      # pure: physics step, spawner, collision, scoring – test surface  
  engine.js    # loop/camera/parallax/input (generic – 07 forks this file)  
  render.js    # draw calls: world, player, obstacles, HUD (procedural art)  
  main.js      # state machine + glue  
  config.js    # flags + strings (host-overridable); tuning constants source of truth  
  storage.js   # guarded facade: versioned best/prefs + migrations, remote adapter slot
```

`core.js` exports: `createPlayer()`, `stepPlayer(player, dt, input)` (jump/buffer/coyote/gravity), `createSpawner(rng)`, `nextSpawn(spawner, speed, lastX)` (fairness rule), `aabbHit(playerBox, obsBox, inset)`, `meters(px)`, `tierFor(m)`. `engine.js`: `createLoop` (from `shared/loop.js`), camera/parallax scroll, `visibilitychange` wiring. `render.js`: pure draw functions taking state (no logic). `main.js`: ``menu/playing/paused/gameover`` transitions.

7. Edge cases

1. Backgrounded mid-air → pause freezes physics; resume continues the same arc exactly.
2. 60 Hz vs 120 Hz displays → identical trajectories (fixed step; render reads sim state).
3. Frame spike > 250 ms → clamped (no tunneling through obstacles).
4. Jump pressed during gameover/menu → ignored (input only in `playing`).
5. ❤️ held + fatal hit → heart consumed, invulnerability ignores hits for exactly 1 s (no double-consume), HUD heart count updates.
6. Storage disabled → no best score, playable.
7. Tab-crash recovery: no mid-run persistence (stated; run is lost — acceptable).

8. Testing plan (core.js, node:test / test.html)

- Jump arc: from ground, max jump clears a 90 px tall barrier at min speed 320 px/s.
- Jump-cut: release at 50 % rise → apex lower than full hold (assert heights).
- Spawner sweep: for speed 320→900 step 20, 500 spawns each — no gap < `minGap(speed)`; doubles' inner spacing within 140–180 px.
- Collision: inset math (corner cases: 1 px overlap inside inset = no hit; at box edge = hit).
- Scoring: meters conversion; +10 awarded exactly once per obstacle.

9. Task breakdown

P0 — playable core

- ☐ `core.js` physics + spawner + collision + scoring (+ tests §8)
- ☐ `engine.js` loop, ground scroll, camera; player jump with coyote/buffer/cut
- ☐ Hurdles, speed ramp, tiers, game over; menu/gameover states; best distance

P1 — full rules & feel

- ☐ Parallax layers; procedural player run/jump animation (shapes); landing dust
- ☐ Tall barrier + double hurdles; tier announcements; WebAudio jump/land/crash + mute
- ☐ Pause overlay + `visibilitychange` (shared loop handles)

P2 — persistence/share/QA

- ☐ ❤️ pickup; share (`Ran {m} m - beat that!`); `?debug=1` hitboxes
- ☐ QA gate (master README §5): 60 fps phone check, offline, isolation greps

P3 — polish

- ☐ Day/night palette shift per 500 m; slide/duck obstacle; fast-fall; gamepad

10. Acceptance criteria

- §8 tests green; no impossible spawns (automated sweep proof).
- Identical jump arcs at 60/120 Hz (sim log comparison).
- Death never ambiguous: `?debug=1` boxes match shipped hitbox math exactly.
- Restart ≤ 1 tap, < 300 ms to first obstacle; 60 fps on a mid-range phone with particles on.

11. Deferred coupling (intentionally NOT built)

Footer/nav links, analytics events, achievements, leaderboards — per master README §7.

12. Rev 2 architecture upgrade (2026-09-11) — reference: `03-sliding-puzzle.md`

Owner-approved architecture reference for all games (Sliding Puzzle Rev 2). This game is not started, so Rev 2 is **folded directly into the build spec above** (§6 already includes `config.js` + `storage.js`) instead of being retrofitted later. Everything in §1–§11 stands; this section adds the per-game multiplayer analysis and two checklist items.

Multiplayer (per this game's nature)

- **Hot-seat:** alternate runs on one device, compare distance — trivial (a two-slot score line), no backend.

- **Async seeded track — the natural fit:** `createSpawner(rng)` already accepts a seed, so a shared seed gives both players the **identical obstacle sequence**; compare distance. Ship as `?seed=` + a share template carrying the seed (P2).
- **Live racing: gated** — real-time positions need a backend, and competitive timing requires server-side validation (client clocks can't be trusted); needs a \$7 reversal. Not planned.

Checklist additions

- ☒ P0 addition (2026-09-11, Rev 2): `config.js` (flags + per-locale strings, host-overridable) + `storage.js` facade (versioned best/prefs with legacy-key migration, remote adapter slot) — `main.js` rewired onto the facade. (2026-09-12: `config.js` wired into `main.js` — best/no-best/share/toast strings live; legacy-prefs guard fixed; unused `parallax` helper removed.)
- ☒ P2 addition (2026-09-11): Rev 2 storage landed with the facade. `?seed=` support + share template carrying the seed (async challenge) remains queued.
- ☐ Note for 07: Spirit Runner forks `engine.js` — keep the fork contract in sync when `config.js` / `storage.js` land here. (07 now carries its own `config.js` pattern.)

Game 06 — Flying Snake / Flappy (Complete Plan)

Game 6 of 7 Complexity: ★★ Medium Estimate: 1–2 days Status: Built

Complete plan (supersedes the 2026-09-09 sample). Status: **Built** (P0–P2 complete, 2026-09-11) — full trio in `apps/frontend/public/games/flying-snake/` (`core/render/main`), jest suite + `core.test.html` green; P3 extras (moving pipes, night palette, ghost run) deferred. Slug `flying-snake` ; folder `apps/frontend/public/games/flying-snake/` .
Branding decided: page title "Flying Snake" (master README §8 #8).
Committed 2026-09-11 (owner decision): the folder joined the committed games so its already-committed jest suite stays CI-safe (\$12 Rev 2 pass still pending).

1. Overview

Flappy-style: tap to flap against gravity, thread the gaps. One-tap, instantly restartable, brutally scoreable. Success metric: median session ≥ 8 runs ("one more try" loop confirmed); death always reads as fair.

2. Complete game spec

Physics (fixed timestep 1/120 s, accumulator, 250 ms frame clamp)

Constant	Value	Notes
<code>GRAVITY</code>	1800 px/s ²	
<code>FLAP_VY</code>	−480 px/s	one flap per press (no hold-to-fly)
<code>TERMINAL_VY</code>	+700 px/s	max fall speed
<code>SNAKE_X</code>	30 % width	fixed; world scrolls left
<code>SNAKE_R</code>	14 px	visual radius; hitbox radius ×0.9 (forgiving)
<code>WORLD_SPEED</code>	160 px/s	constant — difficulty scales via gap, not speed

Obstacles (pipe/vine pairs)

- **Gap:** 170 px, shrinks 2 px per point, floor 120 px.
- **Spacing:** 300 px horizontal, constant (density feels fairer than shrinking both axes).
- **Gap center:** random, ≥ 80 px + gap/2 from top and bottom edges.
- **Ceiling grace:** first 2 s of a run the ceiling clamps (no death); ground always lethal.
- **Collision:** circle (×0.9) vs pipe rects + ground strip; circle-rect standard clamp test.
- **Score:** +1 when the snake's x passes a pair's right edge (counted once per pair).
- **Medals:** bronze 10 · silver 20 · gold 40 · platinum 75 (gameover card).

Feel carriers

- **Tilt:** angle = `clamp(map(vy, -480...+700, -25°...+80°))` — the snake reads velocity.
- **Trailing segments:** 5 circles sampled from past positions every 40 ms with a small sinusoidal wiggle — sells "flying snake" with zero art assets.
- Death: white flash 120 ms + 300 ms screen shake + fall animation, then gameover card.

3. Screens & UI

State	Elements
<code>menu</code>	Title, best score + medal, bobbing snake animation, "Tap to start"
<code>ready</code>	World visible + frozen, "Tap to flap" hint; first tap starts physics mid-screen
<code>playing</code>	World + HUD score (large, top-center) + mute toggle
<code>paused</code>	Overlay (auto on <code>visibilitychange</code>)
<code>gameover</code>	Score, best, medal, [ Retry] [ Share]

Virtual viewport 720×960 logical (portrait), scaled/letterboxed, `devicePixelRatio` aware.

`?debug=1` draws hitbox circle + pipe rects (dev aid).

4. Controls

Tap anywhere / Space / ↑ = flap. `pointerdown` based; multi-touch extra flaps within 50 ms ignored; `touch-action:none`. No other input at MVP.

5. Data model (localStorage)

```
game:flying-snake:best  → { score: 23 }
game:flying-snake:prefs  → { muted: false }
```

6. File structure & function inventory

```
flying-snake/
  index.html
  style.css
  core.js    # pure: physics step, spawner, collision, scoring – test surface
  render.js  # procedural draw: snake segments, vines, ground, HUD
  main.js    # state machine (menu/ready/playing/paused/gameover) + glue
```

`core.js` exports: `createSnake()`, `stepSnake(snake, dt, flapped)` (gravity/terminal/flap), `createSpawner(rng)`, `nextGap(spawner, score)` (margins + shrink rule), `circleRectHit(cx, cy, r, rect)`, `tiltFor(vy)`, `medalFor(score)`. `main.js` owns the

run lifecycle: `ready` starts frozen; first flap starts physics; score-on-pass check in the update loop.

7. Edge cases

1. `ready` state: physics frozen, snake bobs at safe y (never inside a spawn line); first flap both starts the run and applies the flap.
2. Backgrounded → pause via shared loop; resume continues the exact fall state.
3. Flap during `gameover` → ignored; retry is a deliberate button (prevents skip-death).
4. Gap center RNG can't place a gap within margins (assert in spawner tests).
5. Score increments once per pair even if the snake weaves back (x only moves forward — snake x is fixed; world moves — so structurally impossible, test asserts it).
6. Storage disabled → no best/medal, playable.
7. Tab-titling: document.title swap on death is P3 fluff — skip.

8. Testing plan (core.js)

- Flap arc: from gap center y, a single flap clears the smallest gap (120 px) without a second flap when centered; two flaps suffice from gap edges (assert math).
- Spawner sweep: 1,000 gaps across scores 0–80 → all margins $\geq 80 \text{ px} + \text{gap}/2$; gap ≥ 120 .
- `circleRectHit`: corner cases (corner clamp), 1 px inside/outside inset behavior.
- `tiltFor`: endpoints (-25° at min vy, $+80^\circ$ at terminal), monotonic.
- `medalFor`: 9/10/19/20/39/40/74/75 boundaries.

9. Task breakdown

P0 — playable core

- ☒ `core.js` physics + spawner + circle-rect collision + score-once (+ tests §8)
- ☒ World rendering: pipes, ground, snake body with tilt + trailing segments
- ☒ menu/ready/playing/gameover states; ceiling grace; death flash/shake; instant retry
- ☒ Best score persistence; pause via `visibilitychange`

P1 — full rules & feel

- ☒ Gap shrink ramp; medals on gameover card; score pop animation
- ☒ WebAudio flap/score/hit + mute; menu bobbing animation

P2 — persistence/share/QA

- ☒ Share (`Flew through {n} gaps – beat that!`); `?debug=1` hitboxes
- ☒ QA gate (master README §5): fps check, offline, isolation greps (browser smoke test at 420×860 covered every state + persistence; a human 10-minute session is still owed before calling it player-proof)

P3 — polish

- ☐ Moving pipes (gentle bob) after score 50; night palette every 25 points; ghost of best run flying alongside `_(deferred)_`

10. Acceptance criteria

- §8 tests green; 1,000-gap sweep proof stored in the test file.
- Deterministic physics across 60/144 Hz (sim log comparison).
- First flap from `ready` never spawns the snake into a pipe.
- Restart ≤ 1 tap, < 300 ms to playable; 60 fps mid-range phone.

11. Deferred coupling (intentionally NOT built)

Footer/nav links, analytics events, achievements, leaderboards — per master README §7.

12. Rev 2 architecture upgrade (2026-09-11) — reference: `03-sliding-puzzle.md`

Owner-approved architecture reference for all games (Sliding Puzzle Rev 2), applied per this game's nature: flappy is score-attack with a seeded spawner, so async "same pipes" challenges are the natural multiplayer shape; the upgrade is persistence hygiene + those challenges. Isolation contract unchanged.

Target additions

```
flying-snake/
  index.html
  style.css
  core.js      # unchanged (physics/spawner/collision/scoring – test surface)
  render.js    # unchanged
  main.js      # unchanged
  config.js    # flags + per-locale strings (share template, medal copy), host-overridable
  storage.js   # guarded facade: versioned best/prefs + migrations, remote adapter slot
```

config.js

Physics constants stay in `core.js` — §2 tables are the spec of record. `config.js` carries flags and per-locale strings: share template, medal names/copy, per-locale UI copy. Host override `window.__FLYING_SNAKE_CONFIG__` → `?locale=` → defaults. `prefs` may override copy but is a cache, not the source of truth.

storage.js facade

Versioned save (`{ version, best, prefs:{muted} }`, migrated from the loose keys) with a remote adapter slot reserved for future account sync — the game never checks auth;

guests keep full local persistence. Mid-run resume: N/A (runs are seconds; death = instant restart by design).

Multiplayer (per this game's nature)

- **Async seeded pipes — the natural fit:** `createSpawner(rng)` already accepts a seed, so `?seed=` + a share template carrying it = the **identical pipe sequence** for a friend; compare score. Pairs naturally with the deferred P3 "ghost of best run".
- **Hot-seat:** alternate runs on one device, compare score — trivial.
- **Live race: gated** — backend + server-side timing + \$7 reversal. Not planned.

Phases

- ☒ R2-1 Hygiene (2026-09-11; strings wired 2026-09-12): `config.js` (locale + share/medal strings, host-overrideable) + `storage.js` (versioned `save` document — best + prefs — migrated from the loose best/prefs keys, corrupt-save sanitization, remote adapter slot); `main.js` slimmed to the facade. 42/42 tests green; flap-spam smoke clean.
- ☐ R2-2 Async seeded challenge: `?seed=` + share template carrying the seed — `createSpawner(rng)` is ready; `main.js` needs to thread the seed through.
- ☐ R2-3 A11y/polish: reduced-motion variant for flash/shake, menu/gameover focus handling, HUD contrast check.

Game 07 — Spirit Runner (Complete Plan)

Game 7 of 7 Complexity: ★★ ★ High Estimate: 5–7 days Status: Not started

Complete plan (supersedes the 2026-09-09 sample). Status: **built 2026-09-11** — Phases A–D complete in `apps/frontend/public/games/spirit-runner/` (`engine.js` forked from Game 05 with slide + swipe grammar, `gates.js` , `core.js` , `render.js` , `main.js`); §8 jest suite (`src/__tests__/games-spirit-runner.test.ts`) and the folder's `core.test.html` harness green. Outstanding: balance pass (§9 Phase D) and the 10-minute manual session (master README §5); P3 deferred below.

1. Overview

Mystical-forest endless runner with light puzzle choices: run, dodge, collect spirit orbs for powers, and at path splits pick the right **rune gate** — wrong pick banishes you to the riskier, richer **shadow realm**. Depth from choices, not controls (one thumb).

2. Complete game spec

Phase A — core run (fork of Game 05)

- Jump (as 05: coyote/buffer/cut) **plus slide**: swipe down / ↓ / S — 600 ms, hitbox drops to 40 % height; slide-jump cancels into a hop.
- Obstacles:

Type	Clearance	Look (procedural)	Introduced
Fallen log	jump	brown rounded rect	0 m
Low branch	slide	green bar at head height	300 m
Spirit guardian (tall)	jump only	glowing wisp column	800 m
Spirit guardian (low)	slide only	hovering wisp disc	800 m
Rune trap	time it	floor spikes pulsing: 1.2 s cycle — 0.7 s dark (safe), 0.5 s lit (lethal)	1200 m

- Speed ramp as 05 (320→900 px/s). **Hearts: 2**; hit = −1 heart + 1.2 s invulnerability (blink); 0 → game over (deliberately softer than 05 — longer-form run).
- Palette shifts by depth tier (dawn → dusk → night forest).

Phase B — orbs & powers

- Orbs spawn in arcs/lines riding the spawner (arc of 5–7 over a jump, line of 4 on flat). Meter: 10 orbs → full. Power = next in a **fixed cycle** (readable): Double jump → Dash → Slow time (durations in the table below).

Power	Duration	Effect
Double jump	8 s	second mid-air jump
Dash	1.5 s	invulnerable, destroys the next obstacle hit (exactly one)
Slow time	4 s	world $\times 0.6$ (audio pitch drops too)

- Power button (bottom-right, thumb reach) lights when meter full; keyboard E; optional auto-trigger setting.

Phase C — rune gates & shadow realm (the twist)

- Every **600 m $\pm$ 100** the path splits into two gates; a hint banner shows 2 s before. `gates.js` (pure) defines **5 rules**:

Rule	Hint text	Correct gate
Parity	"Even spirits walk the left path."	side matching <code>orbsSinceLastGate % 2</code>
Shape echo	"Follow the doubled symbol."	gate whose rune == the previous gate's rune
Negation	"The moon door lies."	NOT the gate matching the banner's shape hint
Sequence	"▲▲▲ ..."	gate continuing the shown 3-symbol pattern
Color trail	"Trust the last light you caught."	side matching last orb's color

- API: `makeGate(ruleId, rng, runState) → { correctSide, hintText, runes }`; `resolveChoice(gate, side) → 'correct' | 'shadow'`. Rules cycle in order per run (learnable), order shuffles per run via seed.
- **Correct** → +100 points, continue forest. **Wrong** → **shadow realm**: 45 s, darker palette, obstacle density $\times 1.5$, orbs worth $\times 2$, guaranteed **+1 spirit shard** on survival; timer HUD countdown; auto-returns to forest (no soft-lock).

Phase D — characters & meta

- **Spirit shards**: `floor(meters/1000) + shadowSurvives + floor(correctGates/3)` per run.
- **Characters** (pick on menu; modifiers):

Character	Unlock	Modifier
Forest Spirit	start	power durations +50 %
Hunter	5 shards	every run starts with Dash charged
Monk	12 shards	Slow time also $\times 0.7$ obstacle spawn rate

- Save persists shards + unlocks forever (best scores separate).

3. Screens & UI

State	Elements
<code>menu</code>	Title, character cards (locked show shard cost), best distance/shards,  Run, mute

State	Elements
playing	World + HUD: hearts, orbs → power meter + button, distance, depth tier
gate	In-world split + hint banner (run continues — no separate screen)
shadow	Palette swap + "SHADOW REALM" banner + shard countdown
paused	Overlay (auto on visibilitychange)
gameover	Distance, orbs, gates correct, shards earned, unlocks, [↺ Retry] [Character] [🔗 Share]

Virtual viewport 900×500 like 05; `?debug=1` draws hitboxes + gate rule id (dev aid).

4. Controls

- Jump: tap / Space / ↑ (double jump while empowered). Slide: swipe down / ↓ / S.
- Power: button (bottom-right) / E; auto-trigger accessibility setting.
- Swipe-up = jump, swipe-down = slide (pointer gesture thresholds: 30 px, 120 ms).
- **Gate choice (built 2026-09-11, plan left it open):** while a split is live the outer thirds of the screen are the doors — tap the left third for the left door, right third for the right (← / → on keyboard); the middle third keeps jumping and slides still work. While the doors are on the track the spawner holds and the spawn cursor jumps past them, so no obstacle ever overlaps the choice window; a split answered by nothing (the doors pass the runner) resolves as `shadow` — the realm takes you, never a soft-lock (\$7.7).

5. Data model (localStorage)

```
game:spirit-runner:save → {
  bestDistanceM: 2340, shards: 17, unlocked: ['spirit'],
  character: 'spirit', settings: { autoPower: false, muted: false }
}
```

6. File structure & function inventory

```
spirit-runner/
  index.html
  style.css
  engine.js  # forked from 05: loop, camera, parallax, input (jump+slide+swipes)
  core.js    # pure: obstacles, orbs, powers, gates resolution, shards – test surface
  gates.js   # pure: the 5 rune rules + hint text
  render.js  # palettes (forest/shadow), glow (lighter composite), particles, HUD
  main.js    # state machine + meta (characters, unlocks)
```

`gates.js` : `RULES` array, `makeGate(ruleId, rng, runState)`, `resolveChoice(gate, side)`,
`hintFor(ruleId, runState)`. `core.js` : `stepPlayer` (jump/slide states), spawner

extensions (spawnOrbArc , spawnTrap , spawnGuardian), applyPower(run, kind) ,
tickPowers(run, dt) (timers incl. character modifiers), shardsFor(run) .

7. Edge cases

1. Gate while a power is active → power timers keep running through the split.
2. Wrong-gate during invulnerability → shadow realm still triggers (it's not damage).
3. Dash inside shadow realm destroys one obstacle only, as in forest.
4. Slow time stacks with Monk modifier → world $\times 0.6$, spawn $\times 0.7$ (multiplicative, capped once each — no double application).
5. Shadow realm timer always expires exactly (fixed-step accumulation; pause-safe); pays shards exactly once.
6. Unlocks persist across character switching; switching mid-menu only (never mid-run).
7. Rule sequence: 5 rules cycle; run seed shuffles the order; no rule repeats within a cycle; gates ≥ 600 m apart never overlap the shadow realm window (spawner defers).
8. Storage disabled → unlocks lost per session, playable.

8. Testing plan (gates.js + core.js)

- All 5 rules: makeGate + resolveChoice → correct side deterministic given a fixed runState (table tests per rule, 20 rng seeds each).
- Hint text present and never empty for any rule/state.
- Shard math: boundaries at 999/1000 m, 1/2/3 correct gates, shadow survive.
- Power timers: durations incl. Forest Spirit $\times 1.5$; Dash single-destroy assertion.
- Shadow realm: density multiplier applied; timer expiry; single shard payout.
- Spawner: gate deferral rule (§7.7) over 1,000 simulated runs.

9. Task breakdown

Phase A — core run (≈ 2 days)

- ☒ Fork engine from 05; slide + low-branch/log/guardian/trap obstacles; trap pulse cycle
- ☒ 2 hearts + invulnerability; depth palettes; pause/blur; menu/gameover; best distance

Phase B — orbs & powers (≈ 1.5 days)

- ☒ Orb arcs in spawner; power meter + cycle; DJ/Dash/Slow implementations
- ☒ Power button + auto-trigger setting; WebAudio (collect/power/hit) + mute

Phase C — rune gates & shadow realm (≈ 2 days)

- ☒ gates.js 5 rules + hints (+ tests); split-path rendering; resolution flow
- ☒ Shadow realm: palette, density $\times 1.5$, $\times 2$ orbs, shard payout, timed return, banner

Phase D — characters & meta (≈ 1.5 days)

- ☒ Shard earning; unlock thresholds; 3 character modifiers; picker UI

- ☐ Balance pass: median first-run 30–45 s, each gate rule learnable ≤ 3 exposures (analytically tuned — obstacle intro at 0/300/800/1200 m, splits 600 ± 100 m, hearts 2 — but no human playtest yet)
- ☒ Share (`Ran {m} m as {character}... {pts}`); QA gate (master README §5) — automated parts done (§8 suites green, offline static, isolation greps clean); the 10-minute manual phone-width session remains for the owner

P3 — polish

- ☐ Sprite/hand-drawn art slot (post-MVP); music track; daily depth seed; extra rules
- ☐ Achievements/leaderboards: only if isolation decision is reversed (master §7)

10. Acceptance criteria

- Each phase ships playable alone (A = good runner; A+B = powered; +C = full twist; D = meta).
- §8 tests green — all gate rules proven deterministic per state.
- Shadow realm never soft-locks; pays exactly once; always exits.
- 60 fps mid-range phone with particles + glow on; pause never desyncs timers.
- One-thumb: jump, slide, power reachable at 360 px width.

11. Deferred coupling (intentionally NOT built)

Footer/nav links, analytics events, platform achievements integration, leaderboards — per master README §7 (revisit only if the owner reverses isolation).

12. Rev 2 architecture upgrade (2026-09-11) — reference: `03-sliding-puzzle.md`

Owner-approved architecture reference for all games (Sliding Puzzle Rev 2), applied per this game's nature: spirit runner already has seeded, choice-rich runs (the run seed shuffles gate-rule order), so async "same seed" challenges compare runs fairly; the upgrade is persistence hygiene + config extraction. Isolation contract unchanged.

Target additions

```
spirit-runner/
  index.html
  style.css
  engine.js      # unchanged
  core.js        # unchanged (obstacles, orbs, powers, shards – test surface)
  gates.js       # unchanged (5 rune rules)
  render.js      # unchanged
  main.js        # unchanged
  config.js      # flags + per-locale strings (share template; gate hints stay in
                  # gates.js – rule-coupled), host-overridable; §2 tuning constants stay
                  # in core.js (moving them is deferred until the balance settles)
  storage.js     # guarded facade: versioned `game:spirit-runner:save` (shards, unlocks,
                  # character, settings) + migrations + remote adapter slot (created
                  # 2026-09-12, extracted from main.js)
```

Why versioning matters here

This is the composite save most likely to need a migration later (shards, unlocks, character, settings in one document) — version it **before** players accumulate shards, and migrate on read.

config.js

The §2 tuning tables (power durations, densities, shard formula) remain the spec of record; `config.js` is where the shipped values live so balancing passes (§9 Phase D) are data edits. Per-locale strings for gate hints, share template, character copy. Host override `window.__SPIRIT_RUNNER_CONFIG__` → `?locale=` → defaults. `prefs` may override `config` but is a cache, not the source of truth.

Multiplayer (per this game's nature)

- **Async seeded run — the natural fit:** share the run seed → identical gate-rule order and spawner stream; compare distance/shards. Gate choices stay the human skill, so the comparison is fair without a server.
- **Hot-seat:** alternate runs on one device, compare shards/distance — trivial.
- **Live co-op/race (incl. shadow realm): gated** — backend + server timing + §7 reversal. Not planned.

Phases

- ☒ R2-1 Persistence hygiene (2026-09-11; completed 2026-09-12): `config.js` (locale + share template + the host-injectable `remoteAdapter` seam; host-overridable), the versioned save extracted to `storage.js` (writes carry `version: 1`; load normalizes per field), and the tall-guardian gap fixed per §2 (spawner emits `h = GUARDIAN_TALL_H`; `obstacleBox` full-height jump-only branch). **Deferred:** moving the §2 tuning constants into `config.js` — they live in `core.js` and are imported by every module; a dedicated pass when the balance settles. Gate-hint copy stays in `gates.js` (rule-coupled). 43/43 tests green.
- ☐ R2-2 Async seeded-run challenge (`?seed=` + share template carrying it).

- ☐ R2-3 Still owed from §9 Phase D: the balance pass and the owner's 10-minute manual session — unchanged by Rev 2.

Source: plan/games markdown · generated 2026-09-09 · part of the 2D Games sample plans (plan/games/pdf).